

Unified Machine Learning Lifecycle Management: A Platform Architecture for Scalable, Governed, and Reproducible ML Systems

Adwaita Janardhan Jadhav

August 2022

Abstract

Production machine learning systems increasingly consist of heterogeneous infrastructure spanning data ingestion, feature computation, distributed training, evaluation, model registration, deployment, observability, and continuous retraining. In many organizations these capabilities evolve independently and are connected through model-specific pipelines. As the number of models increases, this architecture produces pipeline proliferation, duplicated infrastructure logic, inconsistent evaluation practices, fragmented lineage, and substantial operational overhead. Each new model tends to add a new set of bespoke infrastructure artifacts rather than reusing capabilities that already exist elsewhere in the organization.

This paper presents a Unified Machine Learning Lifecycle Management Platform, an architectural approach that treats the complete machine learning lifecycle as a reusable platform abstraction rather than a collection of independently authored pipelines. The architecture introduces four core ideas: metaworkflows for configuration-driven workflow generation; separation of the ML control plane from the execution plane; first-class lineage and lifecycle state management; and policy-driven evaluation and deployment governance. Under this model, machine learning engineers specify model behavior, datasets, resources, evaluation requirements, and deployment policies through declarative configuration and stable software interfaces. The platform translates these specifications into executable lifecycle graphs while automatically managing orchestration, metadata, lineage, observability, validation, and governance.

The resulting architecture shifts the fundamental scaling unit of an ML organization from the individual pipeline to the reusable platform capability. Instead of constructing new infrastructure for every model, organizations extend common lifecycle primitives that support many heterogeneous workloads. We describe the architecture in detail, analyze its scaling characteristics relative to pipeline-centric designs, discuss implementation considerations including metadata storage, idempotent execution, multi-tenancy, and migration, and identify limitations and open problems. The approach enables reproducibility, reduces workflow duplication, standardizes production controls, and provides a scalable foundation for operating large numbers of machine learning models.

Keywords: *Machine Learning Infrastructure; MLOps; ML Lifecycle Management; Workflow Orchestration; Metaworkflows; Model Governance; Model Lineage; ML Platform; Distributed Training; Model Deployment; Reproducibility*

1. Introduction

The complexity of developing a machine learning model is fundamentally different from the complexity of operating that model reliably in production. A research workflow may require only a dataset, a model implementation, a training loop, and an evaluation metric. A production machine learning system, by

contrast, depends on a substantially larger infrastructure ecosystem. Sculley et al. [1] characterized this gap in terms of technical debt: the learning algorithm is a small fraction of the code and operational surface of a deployed system, and the surrounding infrastructure dominates long-term cost.

The infrastructure surrounding a production model typically includes data ingestion and validation, dataset versioning, feature computation, distributed model training, experiment tracking, model evaluation, artifact storage, model registration, deployment infrastructure, runtime monitoring, lineage tracking, access control, governance, failure recovery, and periodic model refresh. Each of these concerns has its own failure modes, its own operational requirements, and its own evolution over time.

In many organizations these systems are developed incrementally. A team building its first production model creates a training workflow. Another team creates a separate evaluation pipeline. Deployment infrastructure is subsequently attached to both. New models reuse portions of this infrastructure while introducing model-specific modifications. The result is an architecture in which every model owns a private vertical stack of infrastructure, as illustrated in Figure 1.

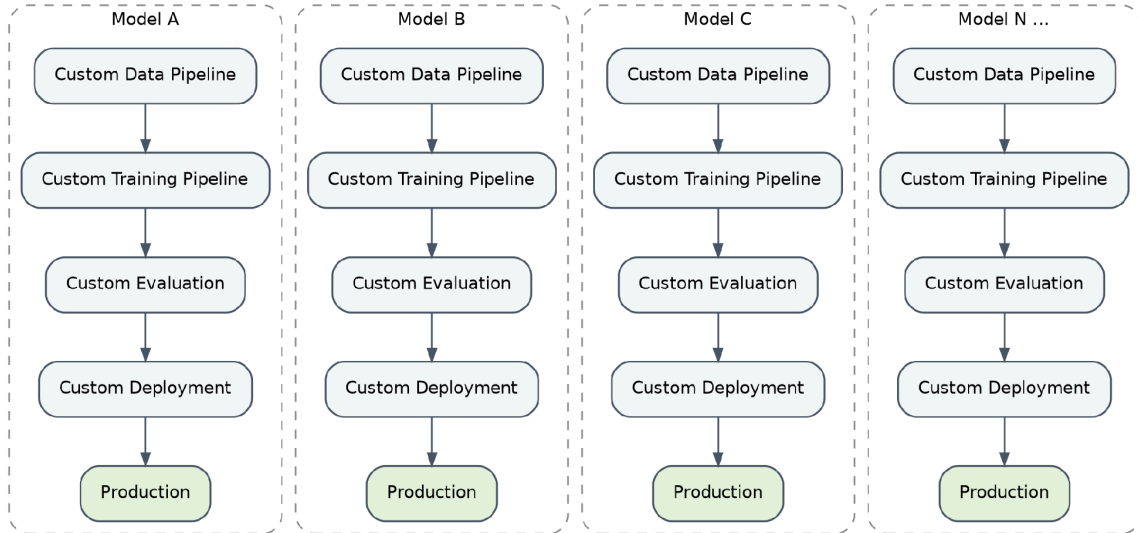


Figure 1. Pipeline-centric ML infrastructure. Each model owns a private vertical stack of data, training, evaluation, and deployment logic. Structural similarity across stacks is not exploited.

Although these pipelines frequently perform structurally similar operations, they become separate engineering artifacts with independent ownership, configuration, dependencies, validation logic, and operational behavior. We refer to the resulting condition as pipeline proliferation: infrastructure complexity grows at least linearly with model count, and often faster, because each pipeline also introduces its own integration points with shared systems.

This paper proposes a different architectural model. In a Unified Machine Learning Lifecycle Management Platform, the complete machine learning lifecycle is represented as a reusable platform abstraction. Rather than requiring engineers to construct a pipeline for each model, the platform provides standardized lifecycle primitives from which model-specific executions are generated. The architectural inversion can be stated simply:

Traditional ML infrastructure: New model leads to a new pipeline.

Unified lifecycle infrastructure: New model leads to a configuration of an existing lifecycle abstraction.

The distinction has significant implications for scalability, reproducibility, governance, and engineering productivity. It changes what an ML engineer writes (intent rather than mechanics), what the platform team maintains (capabilities rather than pipelines), and how the organization reasons about risk (policies enforced by construction rather than by review).

1.1 Contributions

This paper makes the following contributions:

1. We characterize pipeline-centric ML infrastructure and identify four systemic failure modes: workflow proliferation, infrastructure duplication, inconsistent governance, and coupled infrastructure (Section 3).
2. We present a layered architecture that separates the ML control plane (semantic state, policy, lineage) from the execution plane (computation), connected by a metaworkflow layer that generates executable lifecycle graphs from declarative specifications (Sections 5 through 9).
3. We define a first-class object model for datasets, feature sets, training runs, models, evaluations, and deployments, and show how end-to-end lineage and reproducibility follow directly from that model (Sections 10 through 14).
4. We describe evaluation, lifecycle state, observability, failure recovery, governance, and continuous refresh as platform primitives rather than model-specific scripts (Sections 12 through 18).
5. We analyze the scaling behavior of the architecture relative to pipeline-centric designs and discuss implementation and migration considerations, limitations, and open problems (Sections 20 through 24).

1.2 Organization

Section 2 reviews background and related systems. Section 3 analyzes the limitations of pipeline-centric infrastructure. Section 4 states design goals. Sections 5 through 19 present the architecture component by component. Section 20 analyzes scaling. Section 21 contrasts infrastructure before and after lifecycle unification. Section 22 discusses implementation considerations. Sections 23 and 24 discuss broader implications and limitations. Section 25 concludes.

2. Background and Related Work

2.1 Workflow Orchestration

General-purpose workflow orchestrators such as Apache Airflow, Argo Workflows, and Kubeflow Pipelines represent computation as a directed acyclic graph (DAG) of tasks. The orchestrator resolves dependencies, schedules execution, records task-level state, and performs retries. These systems are

mature and widely deployed. However, they are deliberately agnostic to what the tasks compute. An orchestrator knows that Task B depends on Task A; it does not know that Task A produced a dataset version and Task B produced a candidate model that must be compared against a production baseline before promotion. The semantics of machine learning live above the orchestrator, in whatever code the pipeline author writes.

2.2 Integrated ML Platforms

Several organizations have described internal platforms that add ML-specific structure on top of orchestration. TFX [2] provides a set of standard components (data validation, transform, trainer, evaluator, pusher) with typed artifacts and a metadata store. Uber's Michelangelo [3] and Facebook's FBLearner Flow [4] describe end-to-end platforms covering feature management, training, evaluation, and serving. Netflix's Metaflow [5] emphasizes a Python-native developer experience with versioned artifacts and transparent scaling. Open-source tools such as MLflow [6] address experiment tracking, model packaging, and registry functions.

These systems share the insight that ML needs an abstraction above generic orchestration. The architecture described in this paper is aligned with that lineage and differs primarily in emphasis. We treat the lifecycle itself, rather than a component library, as the unit of reuse; we make the generation of workflows from configuration (metaworkflows) an explicit architectural layer; we elevate lifecycle state and policy enforcement to the control plane; and we treat governance as a property that models inherit by construction rather than a capability that teams opt into.

2.3 Data and Metadata Management for ML

Polyzotis et al. [7] surveyed data management challenges in production ML, including validation, provenance, and the management of training data as a first-class asset. Systems such as ModelDB [8] and the metadata layer of TFX [2] track experiments and artifacts. Provenance models such as PROV [9] provide general vocabularies for describing how artifacts were derived. Our lineage design (Section 11) draws on these ideas but ties lineage capture to the lifecycle graph so that provenance is recorded as a side effect of execution rather than requiring explicit instrumentation by model authors.

2.4 Testing, Governance, and MLOps Practice

Breck et al. [10] proposed a rubric of tests for production ML readiness spanning data, model development, infrastructure, and monitoring. Amershi et al. [11] and Paleyes et al. [12] documented the software engineering challenges of ML systems and of deployment respectively. Kreuzberger et al. [13] synthesized MLOps principles, components, and roles from literature and practitioner interviews. These works identify what should be verified and governed; the platform described here is concerned with where those checks live. Our position is that they should be encoded once in shared lifecycle stages and evaluation policies (Sections 12 and 17), so that every model passes through them without per-team re-implementation.

3. Limitations of Pipeline-Centric ML Infrastructure

Pipeline orchestration systems are essential components of modern ML infrastructure. However, workflow execution alone does not provide a complete abstraction for managing machine learning

systems. A conventional ML workflow DAG may resemble the sequence in Figure 2: ingest data, generate features, train, evaluate, deploy. The problem is not the DAG itself. The problem is the level at which the DAG is authored.

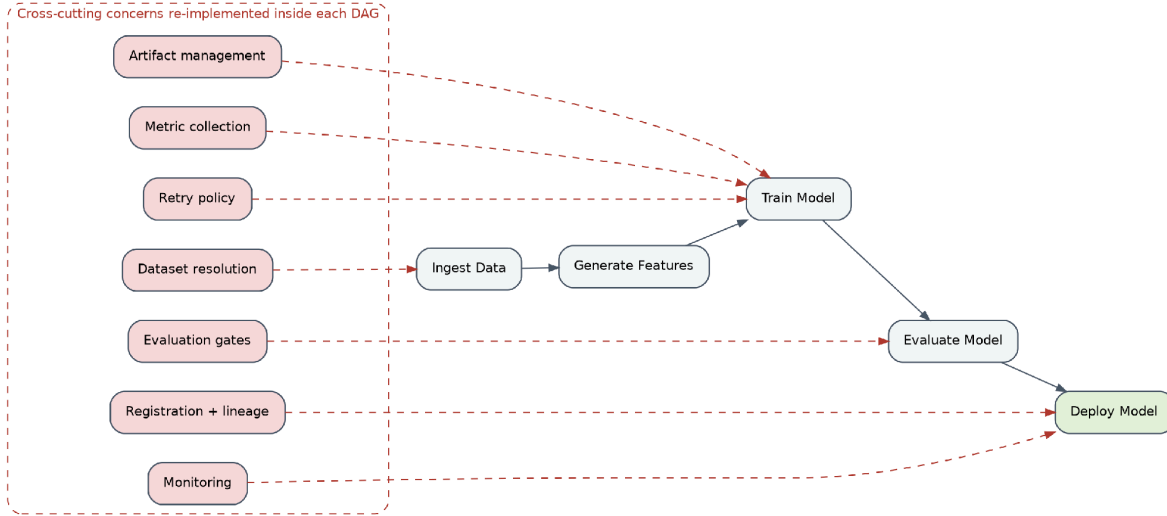


Figure 2. A conventional ML workflow DAG. The visible tasks are only part of the pipeline; the dashed region shows cross-cutting concerns that each model-specific DAG re-implements independently.

If every ML team constructs a model-specific DAG, then each pipeline may independently implement dataset resolution, execution environments, artifact management, metric collection, evaluation gates, retry policies, model registration, deployment integrations, monitoring, and lineage. As an organization grows from tens to hundreds of models, these implementation differences become increasingly difficult to manage. We identify four systemic problems.

3.1 Workflow Proliferation

Structurally similar pipelines are repeatedly implemented for different models. Two ranking models with different features and hyperparameters may share ninety percent of their lifecycle logic, yet exist as two separately maintained pipelines with independent bugs, independent upgrade schedules, and independent on-call ownership. Copy-and-modify is the dominant reuse mechanism, which propagates defects and freezes divergence.

3.2 Infrastructure Duplication

Infrastructure concerns such as artifact persistence, metadata capture, deployment validation, and monitoring become embedded within application-specific workflows. When a shared backend changes (a storage API is deprecated, a scheduler is replaced, a metadata schema evolves), every pipeline that embedded a direct integration must be modified. The cost of infrastructure evolution is therefore proportional to model count rather than to the size of the change.

3.3 Inconsistent Governance

Individual teams implement different evaluation thresholds, approval processes, deployment policies, and lineage standards. Some pipelines compare against a production baseline; others do not. Some record the

dataset version that produced a model; others record only a file path that is later overwritten. Organizational policy (for example, that no model reaches production without a regression check) cannot be enforced because there is no single place where all models pass. Governance becomes a matter of review and trust rather than of construction.

3.4 Coupled Infrastructure

Model implementations become tightly coupled to specific orchestration engines, compute platforms, data systems, or deployment technologies. Training code that calls the scheduler API directly cannot move to a new scheduler without modification. Evaluation code that writes metrics to a particular dashboard backend cannot be reused when the backend changes. The coupling is not usually a deliberate design choice; it accumulates because there is no interface boundary that would prevent it.

Table 1. Comparison of pipeline-centric and unified lifecycle architectures across core operational concerns.

Concern	Pipeline-centric architecture	Unified lifecycle architecture
Unit of reuse	Code snippets copied between pipelines	Lifecycle capability shared by all models
What the engineer authors	Full execution graph and integrations	Declarative specification plus model code
Where evaluation gates live	Inside each pipeline, inconsistently	Control-plane policy applied at qualification stage
Lineage capture	Optional, per-team instrumentation	Automatic side effect of lifecycle execution
Cost of infrastructure change	Proportional to model count	Proportional to size of change
Governance mechanism	Review and convention	Enforced by construction
Failure correlation	Manual, across disparate tools	Automatic, via shared lifecycle context
Marginal cost of a new model	New pipeline plus integrations	New configuration

A scalable ML platform therefore requires a higher-order abstraction: one that captures the lifecycle structure common to all models and exposes model-specific variation as data rather than as code.

4. Design Goals and Principles

The architecture is guided by the following goals. Each goal maps to one or more components described in later sections.

Table 2. Design goals, governing principles, and the architectural components that realize them.

Goal	Principle	Realized by
Reuse over replication	The lifecycle structure is defined once; models vary by configuration.	Metaworkflows (Sec. 6), declarative generation (Sec. 7)
Intent over mechanics	Engineers specify what should happen, not how infrastructure executes it.	Configuration schema (Sec. 7), SDK (Sec. 8)

Goal	Principle	Realized by
Semantic state	The platform knows what a model is, not only which task ran.	Control plane (Sec. 9), object model (Sec. 10), lifecycle state (Sec. 13)
Provenance by default	Every production model has a complete, queryable derivation graph.	Lineage (Sec. 11), reproducibility (Sec. 14)
Quality as contract	Production eligibility is decided by policy, deterministically.	Evaluation primitive (Sec. 12), governance (Sec. 17)
Infrastructure independence	Compute, storage, and orchestration are replaceable behind stable contracts.	Plane separation (Sec. 9), execution profiles (Sec. 19)
Operational economy	Failures are localized, retries are cheap, telemetry is correlated.	Observability (Sec. 15), failure recovery (Sec. 16)
Continuous operation	Retraining is a governed, recurring lifecycle instance.	Continuous refresh (Sec. 18)

5. Unified ML Lifecycle Architecture

The proposed architecture represents machine learning as a managed lifecycle rather than a collection of independent workflows. At the highest level it consists of three layers, shown in Figure 3: a control plane that holds the semantic state of every ML system on the platform; a metaworkflow layer that translates model-level specifications into executable lifecycle graphs; and an execution plane that performs computation.

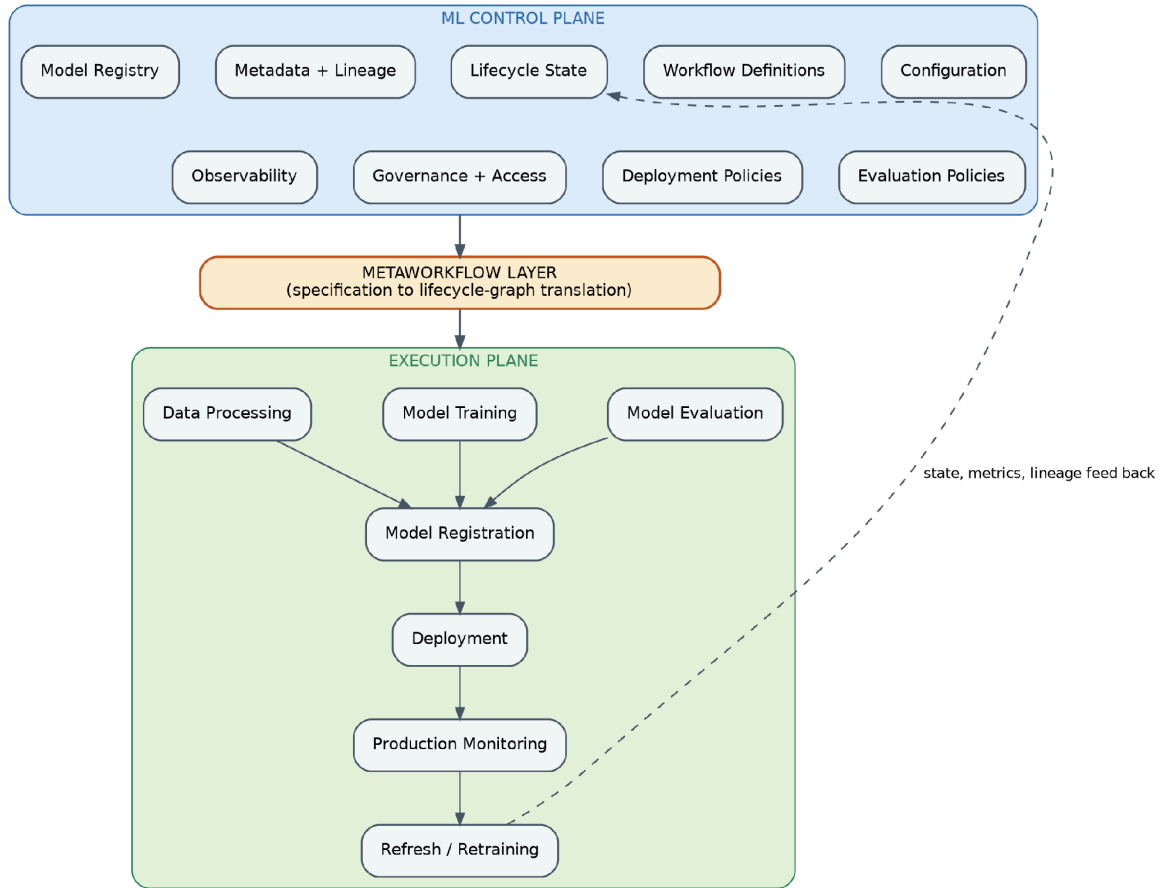


Figure 3. High-level architecture. The control plane holds configuration, state, metadata, lineage, policies, and observability; the metaworkflow layer generates lifecycle graphs; the execution plane runs them and reports state, metrics, and lineage events back to the control plane.

The control plane understands the semantic state of the ML system: which model versions exist, what produced them, where they are deployed, what policies apply, and what is currently executing. The execution plane performs data processing, training, evaluation, packaging, and deployment operations. The metaworkflow layer is the bridge: given a declarative specification and the policies that apply to it, the metaworkflow layer produces a concrete graph that the execution plane can run, and it registers the resulting lifecycle instance with the control plane so that every stage reports back into shared state.

Two properties of this arrangement are worth stating explicitly. First, the control plane is the system of record. Execution engines may be swapped, but the control plane retains lifecycle history, lineage, and policy decisions. Second, the flow is cyclic at the lifecycle level even though each generated graph is acyclic: production monitoring feeds refresh triggers, which instantiate new lifecycle graphs, whose outputs update production state. The remaining sections describe each layer and primitive in detail.

6. Metaworkflows

The metaworkflow is the central architectural abstraction of the platform. A conventional workflow describes one execution graph. A metaworkflow describes the structure of a family of execution graphs.

Instead of authoring Pipeline A, Pipeline B, and Pipeline C as independent artifacts, the platform defines one metaworkflow and instantiates it with Config A, Config B, and Config C, as shown in Figure 4.

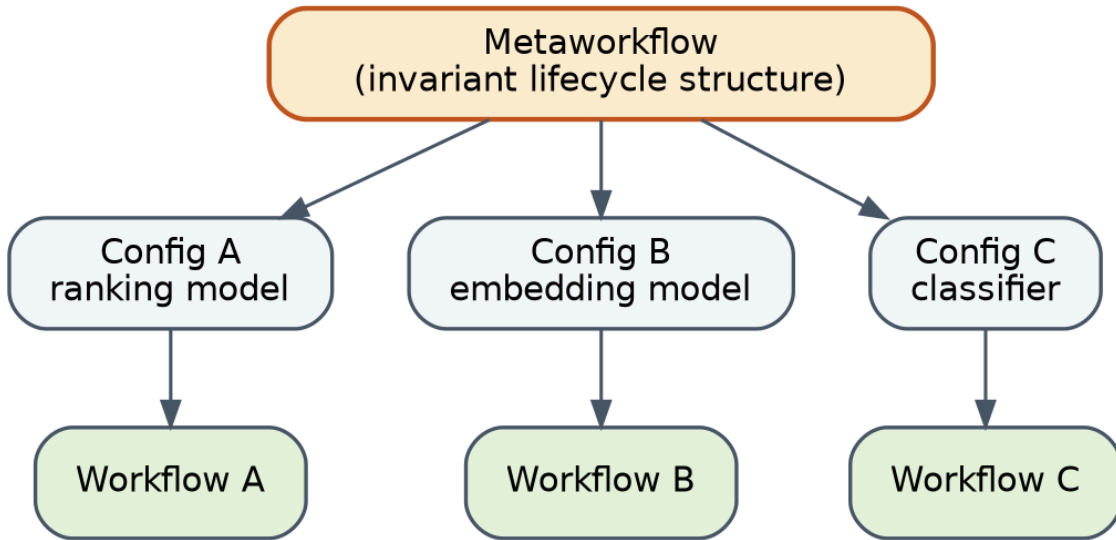


Figure 4. Metaworkflow instantiation. A single metaworkflow defines invariant lifecycle structure; each configuration produces a distinct executable workflow.

6.1 Definition

Formally, a metaworkflow is a function from a configuration space to the space of executable lifecycle graphs. Its inputs are (a) a declarative specification supplied by the model owner and (b) the platform policies in force for the specification's context (team, environment, model class). Its output is a directed acyclic graph of lifecycle stages, each bound to concrete parameters, resources, artifacts, and policy checks. The metaworkflow encodes what must always be true (a candidate is evaluated before it is registered; registration precedes deployment; every stage emits lineage) while configuration encodes what varies (dataset, framework, compute, thresholds, deployment strategy).

6.2 Example Specification

The following specification is representative of what a model owner supplies. It is organized into namespaces that correspond to lifecycle stages.

```

model:
  framework: pytorch
  entry_point: ranking_model.py
  version: v42

data:
  training_dataset: search_training_v81
  evaluation_dataset: search_eval_v17
  features: ranking_features_v23

training:
  compute_profile: gpu_distributed_large
  replicas: 16
  
```

```

hyperparameters:
  learning_rate: 0.0001
  batch_size: 2048
  epochs: 10

evaluation:
  baseline: production_model
  metrics:
    - precision
    - recall
    - ndcg
  qualification:
    require_no_regression: true

deployment:
  strategy: staged
  target: production

monitoring:
  enable_quality_metrics: true
  enable_system_metrics: true

```

The platform converts this specification into the required execution graph. Nothing in the specification names an orchestrator, a cluster, an artifact bucket, a metrics backend, or a registry endpoint. The engineer specifies intent rather than infrastructure mechanics.

6.3 Metaworkflow Families

A platform typically maintains a small number of metaworkflow families rather than a single universal one. Representative families include supervised training with baseline comparison, batch inference and scoring, scheduled refresh, hyperparameter search wrapping an inner training lifecycle, and offline evaluation of an externally supplied model. Families share stage implementations and policy machinery; they differ in graph shape. Adding a family is a platform-team activity performed rarely; instantiating a family is a model-team activity performed continually. Maintaining this asymmetry is what keeps the number of lifecycle definitions small while the number of models grows.

6.4 Extension Points

Metaworkflows must accommodate legitimate variation without collapsing back into per-model pipelines. The architecture provides three controlled extension mechanisms. Stage parameters allow a stage to be tuned through configuration. Stage plug-ins allow a model owner to supply a custom implementation for a stage (for example, a domain-specific evaluator) that conforms to the stage's input and output contract and still participates in lineage and policy enforcement. Optional stages allow a family to include stages that are activated by configuration (for example, a calibration step). What is not permitted is bypassing a mandatory stage or altering the ordering constraints the family defines; those are the invariants that make governance by construction possible (Section 17).

7. Declarative Lifecycle Generation

A major consequence of the metaworkflow model is the separation between workflow definition and workflow instantiation. The lifecycle structure (dataset resolution, data validation, feature generation, training, evaluation, qualification, registration, deployment) remains stable across many models. What

varies are parameters: dataset, feature set, model architecture, training framework, compute resources, hyperparameters, evaluation metrics, acceptance thresholds, deployment environment, and monitoring policy. Figure 5 shows the generation path from specification to executable graph.

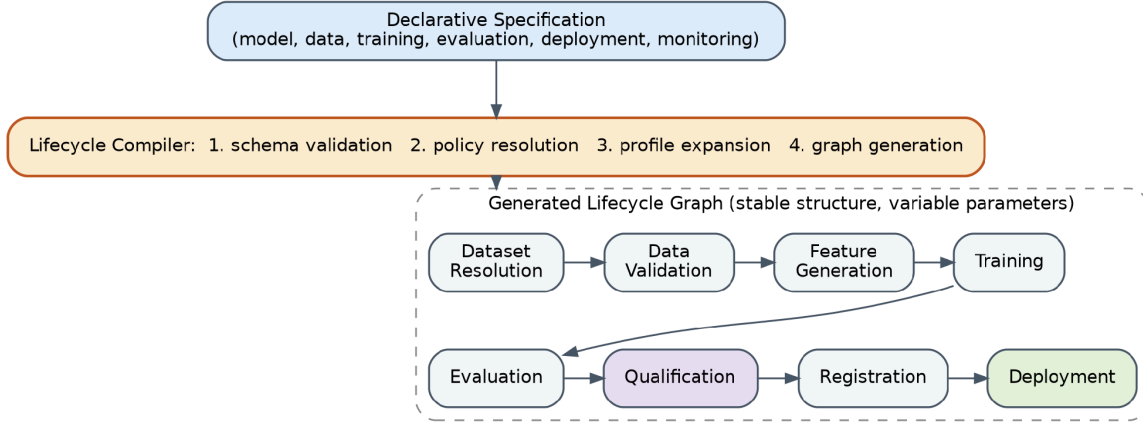


Figure 5. Declarative lifecycle generation. A lifecycle compiler validates the specification, resolves applicable policies, expands execution profiles, and emits a concrete lifecycle graph whose structure is stable and whose parameters vary.

7.1 Configuration Schema

The specification is organized into namespaces. Each namespace is owned by a lifecycle stage and validated against a schema maintained by the platform. Table 3 summarizes the principal namespaces.

Table 3. Principal configuration namespaces and representative fields.

Namespace	Purpose	Representative fields
model	Identifies the model code and framework	framework, entry_point, version, owner
data	Binds versioned datasets and feature sets	training_dataset, evaluation_dataset, features, validation_schema
training	Describes compute and training parameters	compute_profile, replicas, hyperparameters, checkpoint_policy
evaluation	Declares metrics, baseline, and qualification rules	baseline, metrics, thresholds, require_no_regression, slices
deployment	Declares strategy and target environment	strategy, target, rollout_fraction, rollback_policy
monitoring	Enables runtime telemetry and alert policies	enable_quality_metrics, enable_system_metrics, drift_thresholds
refresh	Declares recurring retraining behavior	trigger, schedule, data_window, promotion_policy

7.2 Compilation

Generation proceeds in four phases. Schema validation rejects malformed or incomplete specifications before any compute is consumed; a missing evaluation baseline or an unknown dataset identifier is a compile-time error. Policy resolution merges the organization's mandatory policies (for example, a

minimum regression check or a required approval for production targets) into the specification, so that model owners cannot omit them and do not need to know they exist. Profile expansion replaces symbolic resource names such as `gpu_distributed_large` with concrete scheduler and environment settings (Section 19). Graph generation emits the lifecycle graph with each stage bound to inputs, outputs, resources, and policy checks, and registers the graph with the control plane.

7.3 Consequences

Adding a production model increasingly becomes a configuration operation rather than an infrastructure-development project. The architecture moves from model-specific infrastructure toward shared infrastructure plus model-specific configuration. This changes the economics of operating large ML environments: the fixed cost of the platform is amortized across all models, and the marginal cost of each additional model approaches the cost of writing and reviewing its configuration.

A second consequence is reviewability. A declarative specification is a small, structured document. It can be diffed, reviewed, linted, and versioned alongside model code. Changes to evaluation thresholds or deployment strategy are visible in review as configuration changes rather than being buried in pipeline code.

8. Developer Interface and SDK Layer

Declarative specifications cover the common case, but model development also involves imperative interaction: launching an experiment from a notebook, querying the registry, or composing lifecycle operations in a script. The platform exposes lifecycle capabilities through a stable SDK that serves as the interface between model-development code and platform infrastructure (Figure 6).

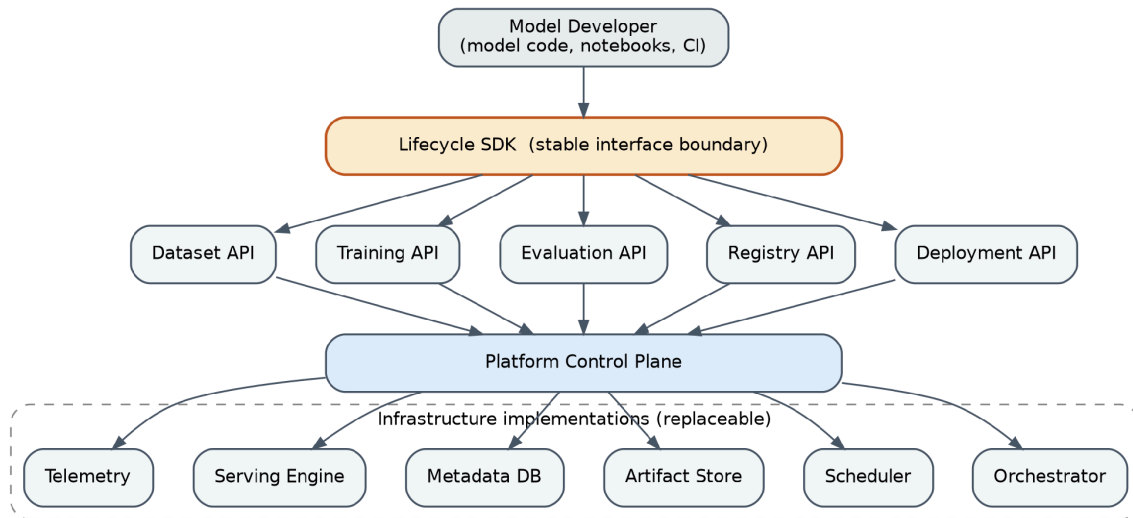


Figure 6. The SDK as an infrastructure boundary. Model developers depend only on lifecycle APIs; the control plane mediates all access to replaceable infrastructure implementations.

Instead of implementing low-level integrations directly, developers invoke standardized platform primitives:

```

model = lifecycle.model(
    name="ranking_model",
    framework="pytorch",
)

training = lifecycle.train(
    model=model,
    dataset="training_dataset_v17",
    compute="distributed_gpu",
)

evaluation = lifecycle.evaluate(
    candidate=training.output,
    baseline="production",
    policy="ranking_quality_gate",
)

lifecycle.deploy(
    model=evaluation.approved_model,
    strategy="staged",
)

```

The conceptual value of the SDK is not syntactic convenience. It creates an infrastructure boundary. Model code depends on the lifecycle interface rather than directly depending on orchestration systems, schedulers, artifact stores, training clusters, metadata databases, deployment engines, or observability backends. Infrastructure can therefore evolve without forcing large-scale modifications to model-development code.

8.1 Equivalence with Declarative Specification

SDK calls and declarative specifications are two surfaces over the same control-plane operations. A sequence of SDK calls produces the same lifecycle objects, lineage edges, and policy checks as the equivalent specification. In particular, `evaluation.approved_model` in the example above is not a plain model reference; it is a reference that exists only if the qualification stage approved the candidate. Deployment of an unqualified model is rejected by the control plane regardless of which surface issued the call.

8.2 Versioning the Contract

Because the SDK is the boundary that absorbs infrastructure change, its own evolution must be disciplined. The platform versions the SDK contract semantically, retains compatibility for deprecated calls for a published period, and records the SDK version used by each lifecycle instance in lineage. This makes it possible to identify which models depend on a deprecated interface and to plan migration by evidence rather than by survey.

9. Control Plane and Execution Plane Separation

A key architectural principle is separation between lifecycle management and workload execution. The two planes and their interaction are shown in Figure 7.

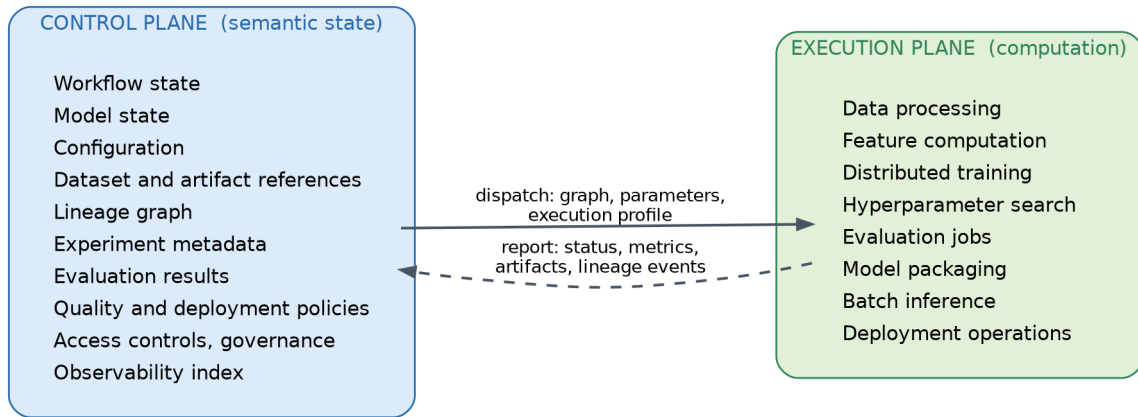


Figure 7. Control plane and execution plane. The control plane dispatches graphs, parameters, and execution profiles; the execution plane reports status, metrics, artifacts, and lineage events.

9.1 Control Plane

The control plane manages workflow state, model state, configuration, metadata, dataset references, artifact references, lineage, experiment metadata, evaluation results, quality policies, deployment state, access controls, governance, and observability. It answers questions such as: Which model version is currently in production? What workflow produced it? Which dataset and feature versions were used? What evaluation allowed deployment? What is the current execution state? Which deployment consumes the model? What failed during the last refresh?

The control plane is deliberately not a compute system. It stores references and state, issues dispatches, and consumes reports. Its availability requirements are those of a metadata service, and its scaling requirements are proportional to lifecycle events rather than to training compute.

9.2 Execution Plane

The execution plane performs data processing, feature computation, distributed training, hyperparameter search, evaluation, model packaging, batch inference, and deployment operations. It receives from the control plane a stage to execute, its bound parameters, and an expanded execution profile; it returns status, metrics, artifact references, and lineage events. Execution engines are pluggable: a stage may be executed by a container job, a distributed training framework, a batch cluster, or a serving system, provided the engine implements the stage contract.

9.3 Portability

This separation provides infrastructure portability. A logical lifecycle may remain unchanged while the underlying compute platform evolves from Platform A to Platform B to Platform C. The lifecycle contract does not have to change, and neither does model code. Migration of the execution plane becomes a platform-team project with a bounded scope rather than an organization-wide rewrite.

10. First-Class ML Objects

The platform treats important ML entities as explicit versioned objects rather than anonymous files produced during execution. Figure 8 shows the core object model and its relationships.

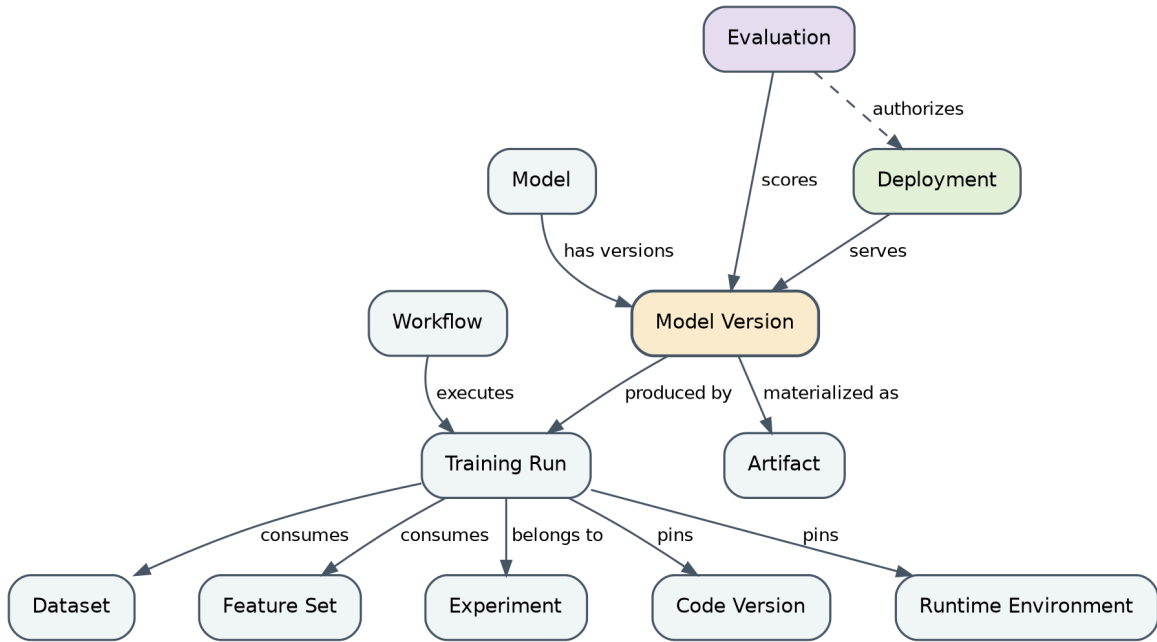


Figure 8. First-class lifecycle objects and their relationships. A model version is produced by a training run that consumes versioned datasets and feature sets and pins code and runtime; evaluations score versions and authorize deployments.

Table 4. Core lifecycle objects, their identity semantics, and key relationships.

Object	Identity	Key relationships
Dataset	Immutable, versioned reference to a data snapshot	Consumed by training runs and evaluations
Feature Set	Versioned feature definitions and computed values	Consumed by training runs; derived from datasets
Experiment	Logical grouping of related training runs	Contains training runs
Training Run	One execution of the training stage	Consumes dataset, feature set; pins code and runtime; produces model version
Model	Named, long-lived entity	Has many model versions
Model Version	Immutable candidate produced by a run	Produced by training run; scored by evaluation; served by deployment
Evaluation	Comparison of a candidate against baseline under a policy	Scores model version; authorizes deployment
Deployment	Placement of a model version in an environment	Serves model version; monitored at runtime
Workflow	One instantiated lifecycle graph	Executes stages; owns training runs, evaluations, deployments

Object	Identity	Key relationships
Artifact	Any content-addressed output	Materializes model versions and intermediate outputs

Representing these entities explicitly enables platform-wide reasoning about ML systems. Queries such as "which production deployments serve a model trained on dataset v112" or "which candidates were rejected by policy ranking_quality_gate last month" become metadata queries rather than forensic investigations. Two design rules keep the object model sound: objects that participate in lineage are immutable once created, and every relationship is recorded by the platform at the moment it is established rather than reconstructed later.

11. Model Lineage

Reproducibility requires substantially more information than a serialized model artifact. A production model should have a complete provenance graph. Figure 9 shows the lineage graph for a representative production model.

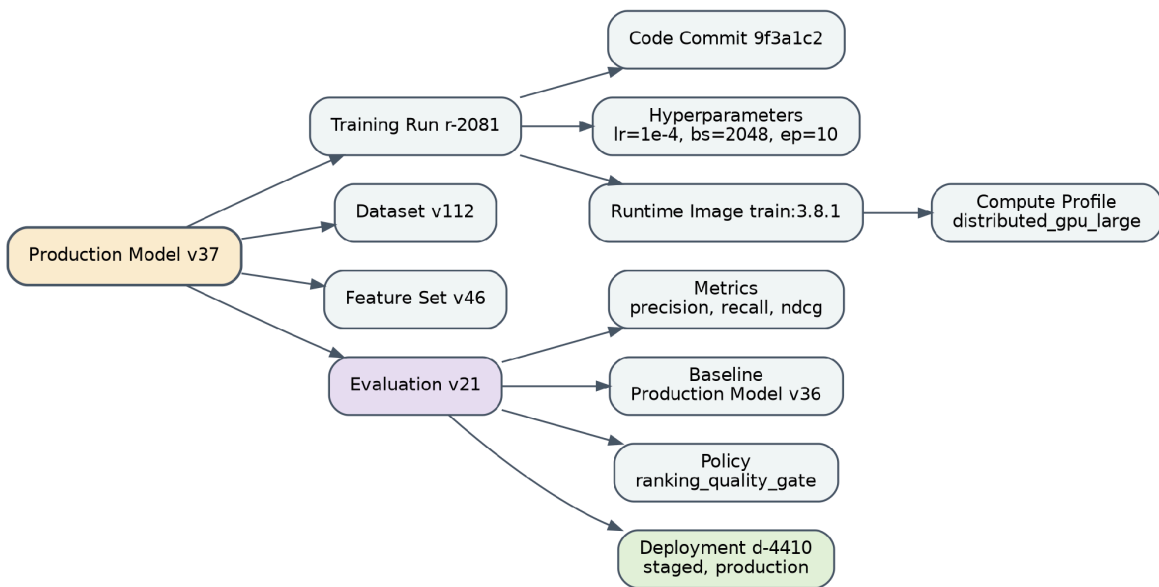


Figure 9. End-to-end lineage for a production model. The graph connects the model to its training run, dataset, feature set, code commit, hyperparameters, runtime image, compute profile, evaluation, metrics, baseline, policy, and deployment. Identifiers shown are illustrative.

With this representation the platform can answer the question "what exactly produced this model?" The answer includes input data, feature definitions, source code, runtime environment, training parameters, model architecture, evaluation results, baseline comparison, qualification policy, and deployment history. Lineage therefore becomes fundamental infrastructure rather than optional experiment metadata.

11.1 Capture Mechanism

Lineage is captured as a side effect of lifecycle execution rather than through explicit instrumentation. Each stage in the generated graph declares its inputs and outputs in terms of first-class objects. When the execution plane reports stage completion, the control plane records the corresponding edges. Because the stage contract requires typed inputs and outputs, a stage cannot complete without declaring what it consumed and produced. Model owners obtain complete lineage without writing lineage code.

11.2 Bidirectional Queries

Lineage supports queries in both directions. Backward queries answer provenance questions (what produced this model). Forward queries answer impact questions (what depends on this dataset). Forward queries are operationally valuable: when a dataset is found to be corrupted or a feature definition is found to be leaking future information, the platform can enumerate every model version, evaluation, and deployment downstream of the defect and initiate remediation systematically.

12. Evaluation as a Lifecycle Primitive

Many ML systems treat evaluation as a collection of scripts executed after training. The proposed architecture instead models evaluation as a first-class lifecycle stage with a deterministic qualification decision (Figure 10).

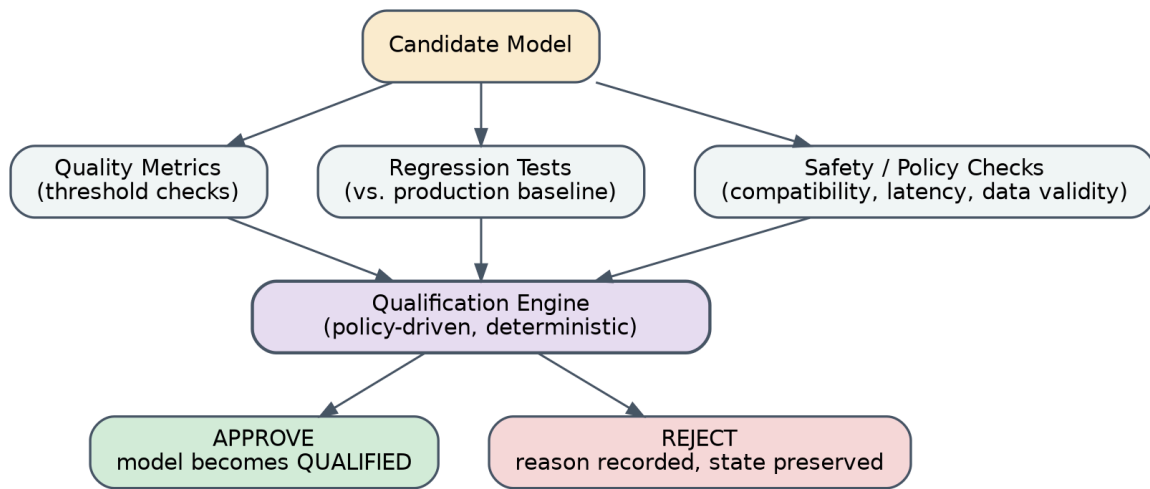


Figure 10. Evaluation and qualification. Quality metrics, regression tests, and safety and policy checks feed a qualification engine that approves or rejects the candidate under a declared policy.

A model does not become production eligible simply because training completes successfully. It must satisfy lifecycle-level qualification requirements, which may include minimum metric thresholds, no regression against baseline, dataset validation, feature validation, model compatibility tests, runtime performance constraints, and deployment-specific policies. This turns model quality from an informal team practice into a platform-enforced contract.

12.1 Qualification Policies

A qualification policy is a named, versioned object in the control plane. It declares which metrics are computed, on which evaluation datasets and slices, how the candidate is compared against which baseline,

what tolerances apply, and which non-metric checks (schema compatibility, latency budget, model size, fairness constraints) are required. A specification references a policy by name; organizational defaults are merged at compile time (Section 7.2). Because policies are versioned, an evaluation records not only its outcome but the exact policy version that produced it. This matters when policies are tightened over time: the platform can identify production models qualified under an older, weaker policy.

12.2 Determinism and Recording

The qualification engine is deterministic: the same candidate, baseline, evaluation data, and policy version yield the same decision. Every decision is recorded with its inputs, and rejections record the specific check that failed. This makes rejection actionable for the model owner and auditable for the organization. A rejected candidate remains in the object model in a REJECTED state; it is not deleted, because rejected candidates are evidence about what did not work.

12.3 Baselines

The baseline for comparison is itself a lifecycle object, most commonly the model version currently in the target deployment. Resolving the baseline at evaluation time, rather than at specification time, ensures that a candidate is always compared against what it would actually replace. For new models with no production predecessor, policies may specify an absolute threshold, a designated reference model, or a shadow evaluation period.

13. Model Registration and Lifecycle State

A model version progresses through explicit states. Figure 11 shows the state machine, including terminal failure and rejection states and a prohibited transition.

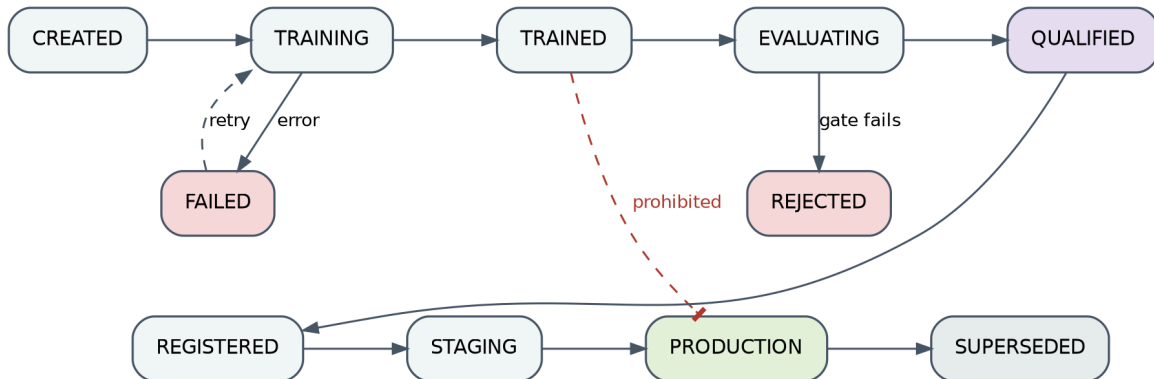


Figure 11. Model version lifecycle state machine. The direct transition from TRAINED to PRODUCTION is prohibited; a version must pass through evaluation, qualification, and registration.

Table 5. Permitted state transitions and the preconditions enforced by the control plane.

From	To	Trigger	Precondition enforced by control plane
CREATED	TRAINING	Training stage dispatched	Dataset and feature set resolved; compute profile expanded

From	To	Trigger	Precondition enforced by control plane
TRAINING	TRAINED	Training stage completed	Artifact registered; code, runtime, and parameters pinned
TRAINING	FAILED	Training stage error	Failure recorded with stage context
TRAINED	EVALUATING	Evaluation stage dispatched	Baseline resolved; policy version fixed
EVALUATING	QUALIFIED	Qualification approved	All policy checks passed; decision recorded
EVALUATING	REJECTED	Qualification rejected	Failing check recorded
QUALIFIED	REGISTERED	Registration stage completed	Registry entry created with lineage reference
REGISTERED	STAGING	Deployment stage dispatched	Deployment policy passed; approvals obtained if required
STAGING	PRODUCTION	Rollout completed	Rollout health checks passed
PRODUCTION	SUPERSEDED	Successor promoted	Successor is in PRODUCTION; rollback target retained

Invalid transitions are prohibited by the control plane, not by convention. A request to deploy a version in TRAINED state is rejected because no qualification decision exists for it. Lifecycle state management creates deterministic operational semantics across models: the same words mean the same thing for every model on the platform, and the same guarantees hold.

13.1 Registration

Registration is the act of making a qualified version discoverable and deployable. The registry entry references the model version, its lineage root, the evaluation that qualified it, the policy version applied, and the owner. Registration is a distinct stage rather than a side effect of training because it is the point at which a candidate becomes an organizational asset with retention, access, and audit implications.

14. Reproducibility

Reproducibility requires capturing the complete execution context. For a training run, the resulting model can conceptually be represented as a function of six inputs (Figure 12).

$$M = f(D, F, C, H, E, I)$$

where D is the dataset version, F the feature definition, C the code version, H the hyperparameters, E the execution environment, and I the infrastructure configuration. Reproducing the model requires preserving all six components. The lifecycle platform automatically records these relationships as part of lineage.

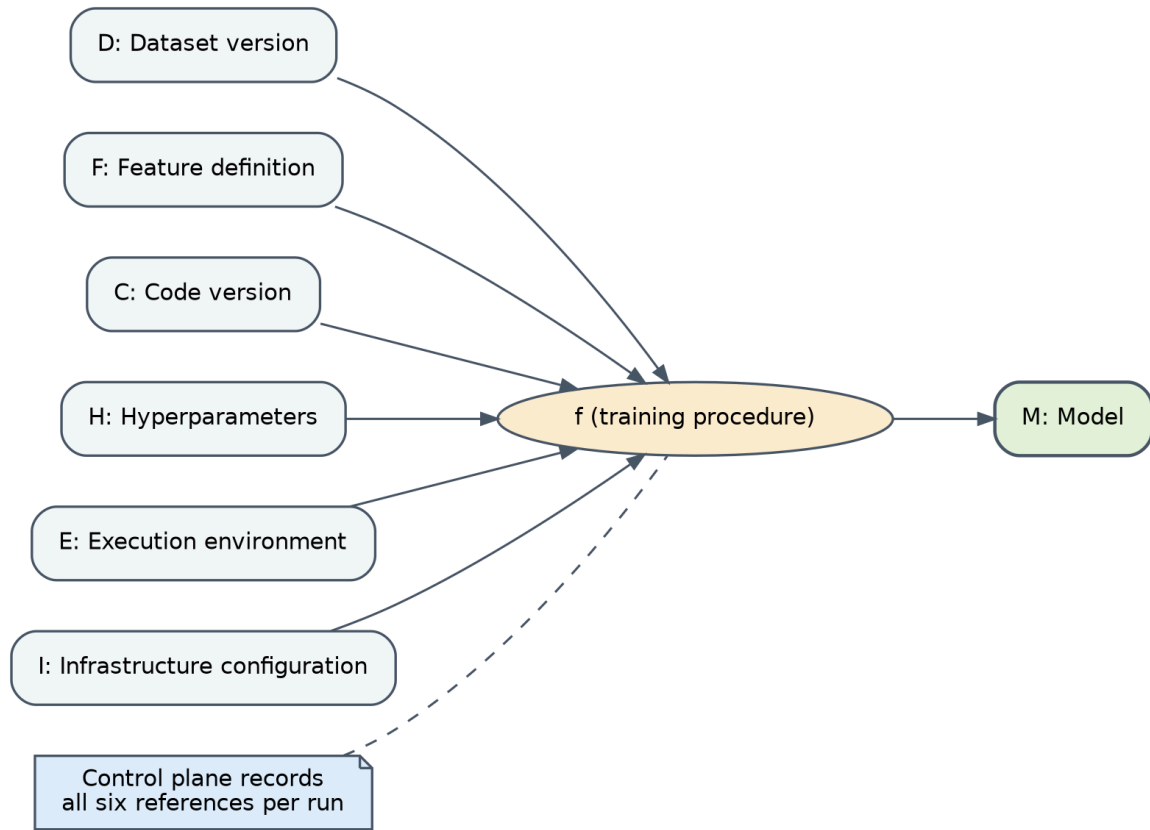


Figure 12. The six inputs that determine a trained model. The control plane records a reference to each for every training run.

This enables a previous execution to be reconstructed from dataset version, feature version, code commit, hyperparameters, container or runtime version, and compute configuration, rather than depending on manually maintained experiment notes.

14.1 Degrees of Reproducibility

Not all six inputs are equally controllable, and it is useful to distinguish levels. Specification reproducibility means the six references are recorded and can be re-dispatched; this is guaranteed by the architecture. Environment reproducibility means the runtime image and dependencies resolve identically; this requires immutable image references and pinned dependency manifests, which the platform enforces through execution profiles. Numerical reproducibility means the trained parameters are bitwise identical; this depends on framework determinism settings, hardware, and the order of distributed operations, and the platform can record the relevant settings but cannot guarantee identity for all workloads. The platform's contribution is to make the first two levels automatic and to make the gap to the third level explicit and measurable.

14.2 Reproduction as a Lifecycle Operation

Because every input is a first-class reference, reproduction is a control-plane operation: instantiate the same metaworkflow with the recorded configuration and pinned references. The reproduced run is itself

recorded with lineage linking it to the original, so that comparisons between original and reproduction are queryable. This capability is what makes incident investigation, audit response, and regression bisection tractable on a platform with many models.

15. Observability

A unified lifecycle architecture enables standardized observability across heterogeneous ML workloads. Telemetry is organized in a hierarchy (Figure 13): lifecycle, workflow, stage, job, task, artifact. Every metric, log line, and event carries the identifiers of all its ancestors, so that any signal can be correlated with the exact workflow, stage, dataset, configuration, and model version involved.

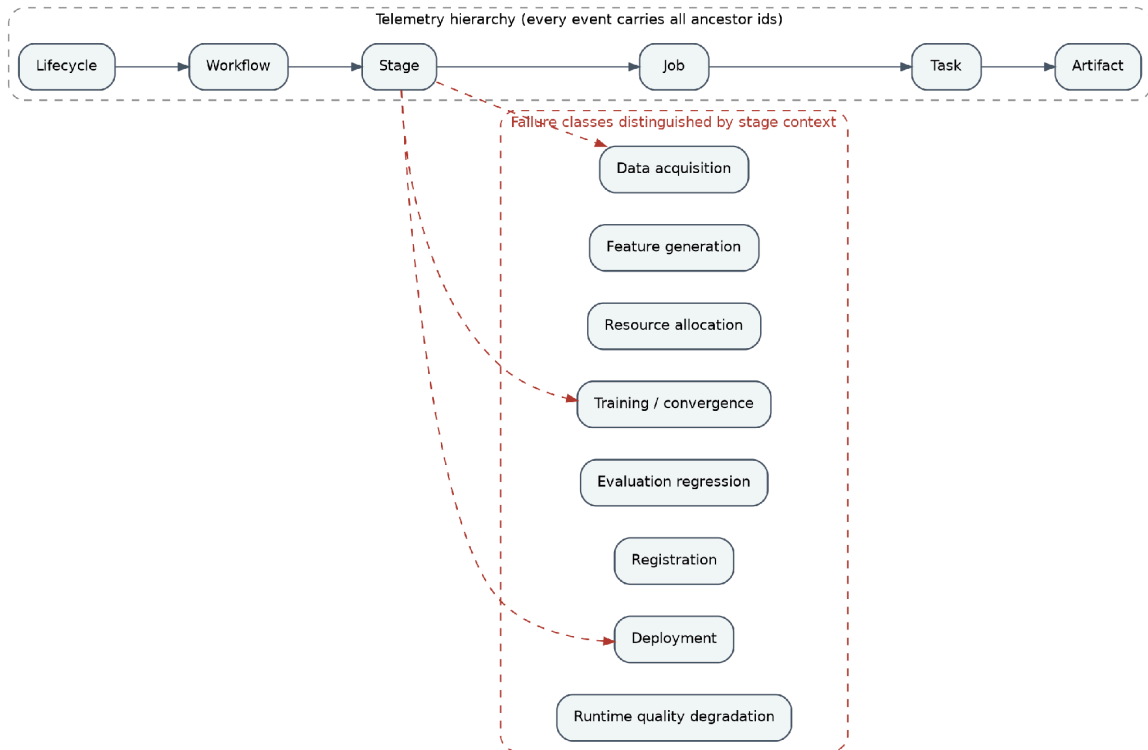


Figure 13. Telemetry hierarchy and failure classes. Because every event carries its full ancestry, failures are attributed to a lifecycle stage automatically rather than being correlated manually across tools.

The platform can distinguish between failure classes that, without lifecycle context, frequently appear in different tools and must be manually correlated. Table 6 lists representative classes and the stage context that identifies them.

Table 6. Failure classes distinguished by lifecycle stage context, with typical signals and platform responses.

Failure class	Stage context	Typical signal	Typical response
Data acquisition failure	Dataset resolution	Missing or unreadable snapshot	Retry stage; alert data owner
Feature generation failure	Feature generation	Schema mismatch, null rate spike	Fail fast; block downstream stages

Failure class	Stage context	Typical signal	Typical response
Resource allocation failure	Training dispatch	Scheduler rejection, quota exhausted	Requeue; adjust profile
Training failure	Training	Process error, checkpoint loss	Resume from checkpoint
Convergence failure	Training	Loss divergence, NaN gradients	Reject run; notify owner
Evaluation regression	Qualification	Metric below baseline	Reject candidate; retain production
Registration failure	Registration	Registry write error	Retry stage only
Deployment failure	Deployment	Rollout health check failed	Automatic rollback
Runtime quality degradation	Production monitoring	Drift, latency, quality decline	Trigger refresh lifecycle

15.1 Quality Telemetry

Observability in an ML platform includes model quality signals, not only system health. The monitoring stage emits prediction distributions, input feature distributions, and, where labels become available, delayed accuracy metrics. These are recorded against the deployed model version so that quality trends are attributable to a specific version and, through lineage, to the dataset and features that produced it. Drift beyond declared thresholds becomes a refresh trigger (Section 18).

16. Failure Recovery and Idempotent Execution

Long-running machine learning workflows are expensive. A pipeline that successfully completes data processing, feature generation, and training but fails during evaluation should not necessarily recompute the preceding stages. The platform therefore maintains persistent stage-level state (Figure 14).

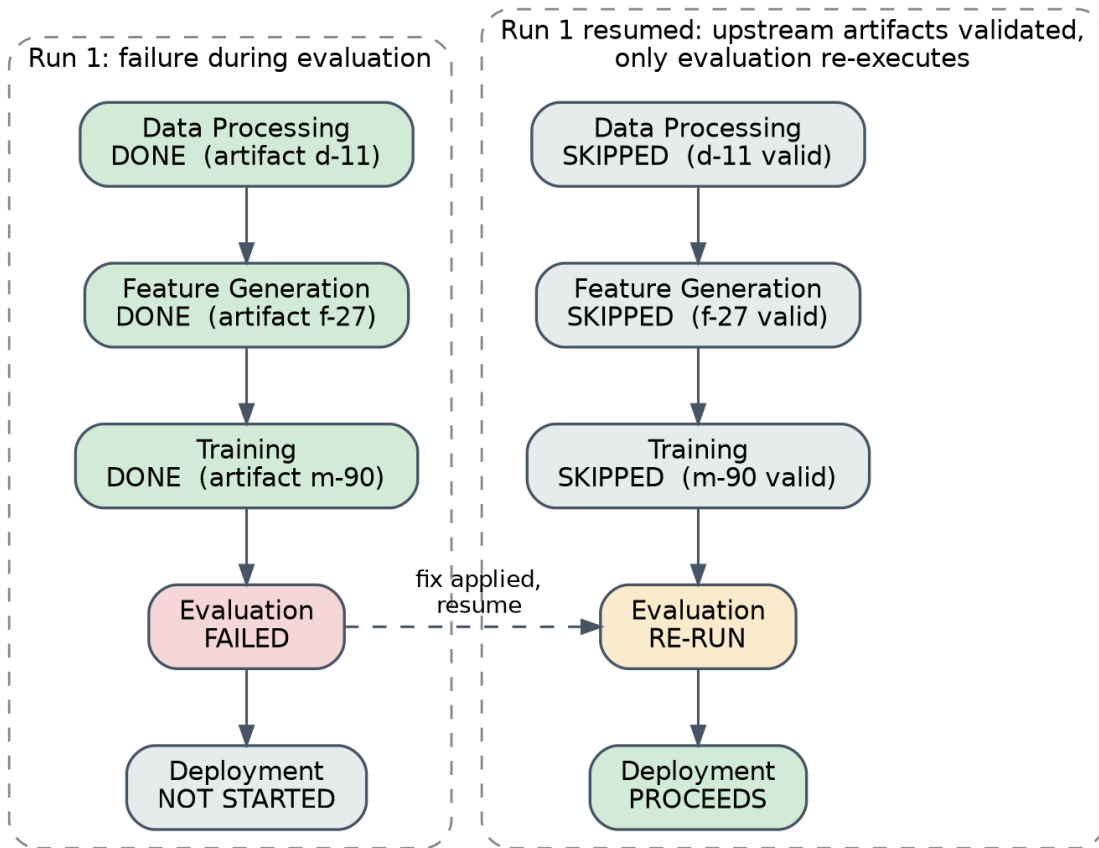


Figure 14. Stage-level failure recovery. After an evaluation failure is resolved, upstream artifacts are validated and only the failed stage re-executes.

After the failure is resolved, evaluation can be executed independently if upstream artifacts remain valid. This requires immutable artifact identifiers, versioned inputs, stage-level state, idempotent execution semantics, dependency resolution, and artifact validity checking. Together these capabilities can substantially reduce wasted compute.

16.1 Idempotency Contract

Each stage is required to be idempotent with respect to its declared inputs: executing the stage twice with identical input references produces equivalent outputs and identical lineage. Stage implementations achieve this by deriving output identifiers from input identifiers and stage version (content addressing), by writing outputs atomically, and by treating an existing valid output as a completed execution. The control plane uses this contract to skip stages whose outputs exist and are valid, whether the skip is due to a retry, a re-run with a partially changed configuration, or a reproduction request.

16.2 Validity

An upstream artifact is valid for reuse if it exists, its content hash matches its recorded identifier, and none of its declared inputs have changed. Validity is determined by the control plane from lineage, not by the stage implementation. When a configuration change touches only the evaluation namespace, dependency

resolution determines that training outputs remain valid and re-executes only evaluation and downstream stages.

16.3 Checkpointing Within Stages

Stage-level recovery does not address failures inside a long training stage. The training stage contract therefore supports checkpoint references as intermediate artifacts. A resumed training stage receives the most recent valid checkpoint as an input, and the checkpoint lineage is recorded so that the final model's provenance includes any resumption events.

17. Governance by Construction

A unified lifecycle platform provides an opportunity to make governance intrinsic to ML execution. Traditional governance frequently occurs after infrastructure is already created: a review board examines a model that has been built, or an audit reconstructs how a model reached production. The unified model reverses this. Every model automatically passes through a governed lifecycle (Figure 15).

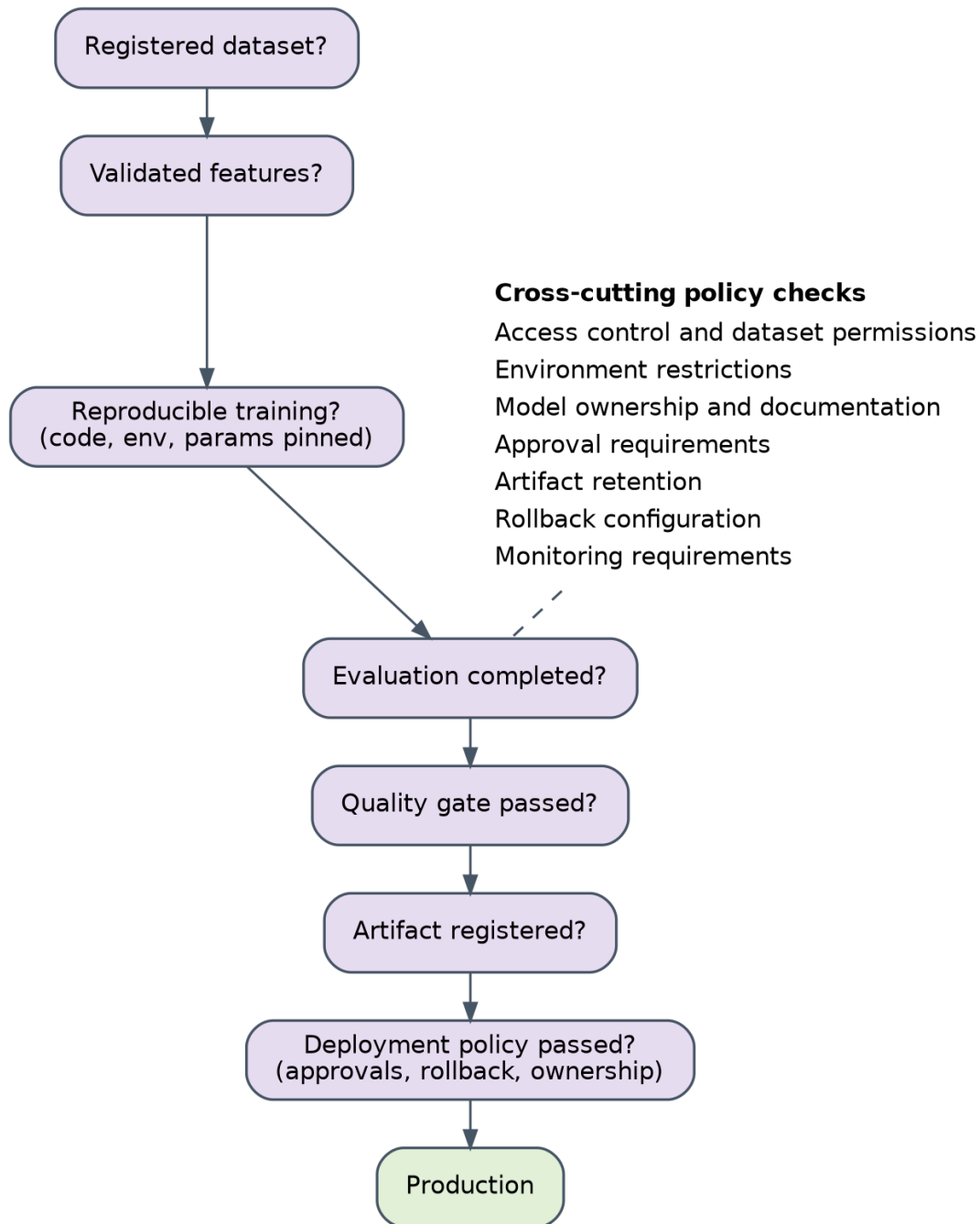


Figure 15. Governance by construction. Every model passes through the same sequence of gates; cross-cutting policy checks are attached to gates rather than to individual pipelines.

Additional policy checks can include access control, environment restrictions, model ownership, approval requirements, artifact retention, dataset permissions, model documentation, rollback configuration, and production monitoring requirements. Because these policies exist in the shared lifecycle, they are inherited automatically by models using the platform. Governance therefore becomes governance by

construction: it is impossible for a model to reach production without having passed the gates, because the only path to production runs through them.

17.1 Policy as Data

Policies are versioned control-plane objects, scoped by organization, team, environment, or model class, and merged at compile time. A policy change (for example, raising a regression tolerance or requiring an additional approver for a class of models) takes effect for every subsequent lifecycle instance without modifying any model's configuration or code. The platform can also report which production models were qualified under superseded policies and, if required, re-evaluate them under the current policy.

17.2 Audit

Because every gate decision is recorded with its inputs and policy version, an audit question of the form "show that model version v37 satisfied all applicable requirements when it was promoted" is answered by a lineage query rather than by document collection. The same mechanism supports regulatory reporting, incident review, and internal model risk management.

18. Continuous Model Refresh

Production models frequently require periodic refresh as data distributions change. Without a unified lifecycle, refresh often requires manually coordinating a new dataset, training, evaluation, and deployment. The lifecycle platform instead represents refresh as a recurring lifecycle instance (Figure 16).

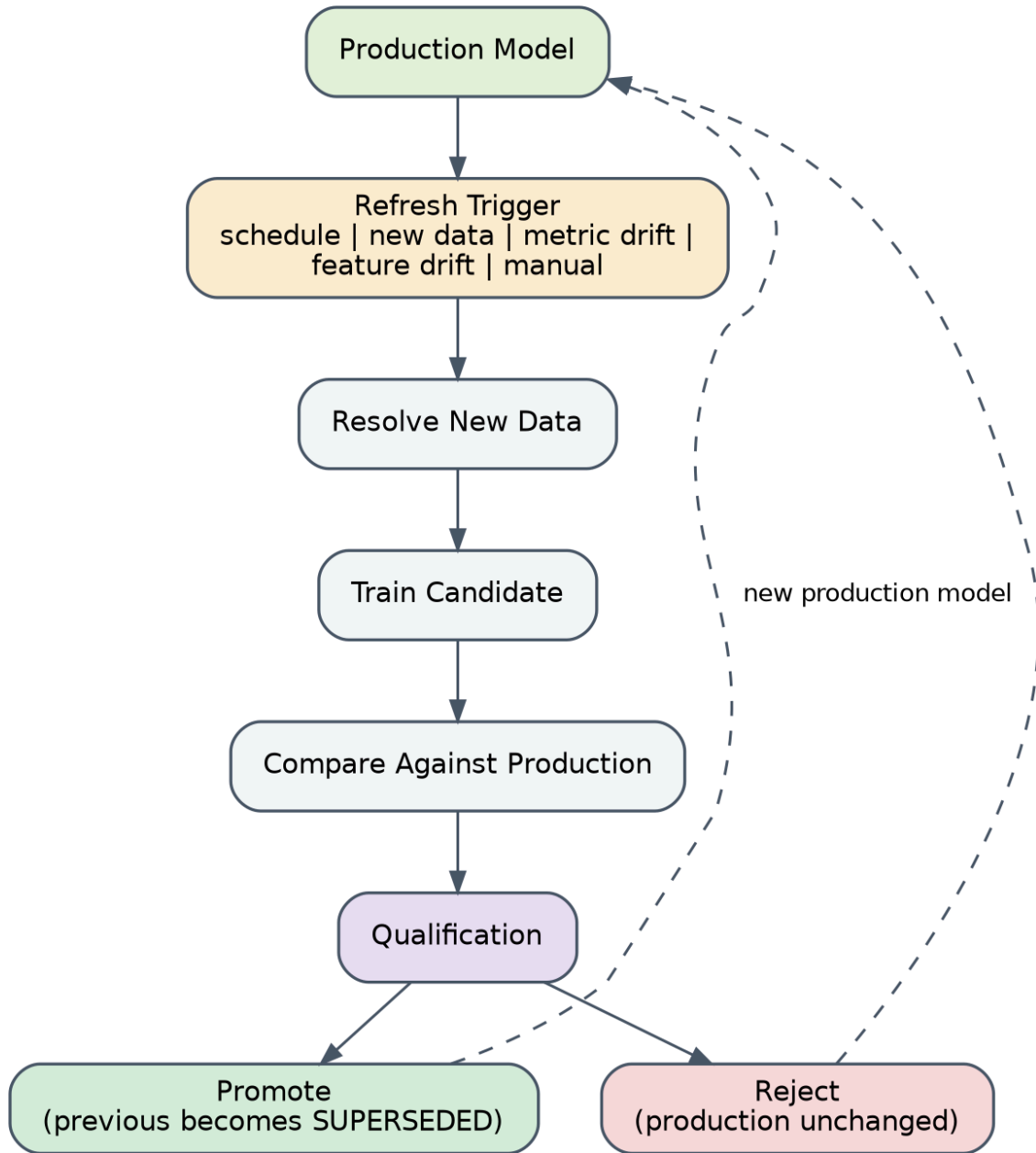


Figure 16. Continuous refresh as a governed loop. A trigger instantiates a lifecycle whose candidate is compared against the current production model and promoted only if qualified.

Table 7. Refresh triggers. All trigger types instantiate the same governed lifecycle.

Trigger type	Source	Behavior
Scheduled	Control-plane scheduler	Instantiate refresh lifecycle at declared cadence
New data arrival	Dataset registry event	Instantiate when a new dataset version satisfying the data window is registered

Trigger type	Source	Behavior
Metric degradation	Production monitoring	Instantiate when quality metrics fall below declared thresholds
Feature distribution change	Production monitoring	Instantiate when input drift exceeds declared thresholds
External request	API or SDK call	Instantiate on demand from an upstream system
Manual	Model owner	Instantiate on demand with optional configuration override

The same quality and governance policies remain applicable. A refreshed candidate that fails qualification leaves the production model unchanged, and the rejection is recorded. A refreshed candidate that is promoted causes the prior version to transition to SUPERSEDED while remaining available as a rollback target. Refresh therefore introduces no new operational path; it reuses the path every model already follows.

19. Multi-Framework and Infrastructure Heterogeneity

Large organizations rarely standardize completely on one ML framework or one class of compute. Different workloads use different frameworks and custom runtimes, and execution may span CPU, GPU, distributed GPU, batch clusters, and specialized accelerators. The unified platform abstracts these environments through execution profiles (Figure 17).

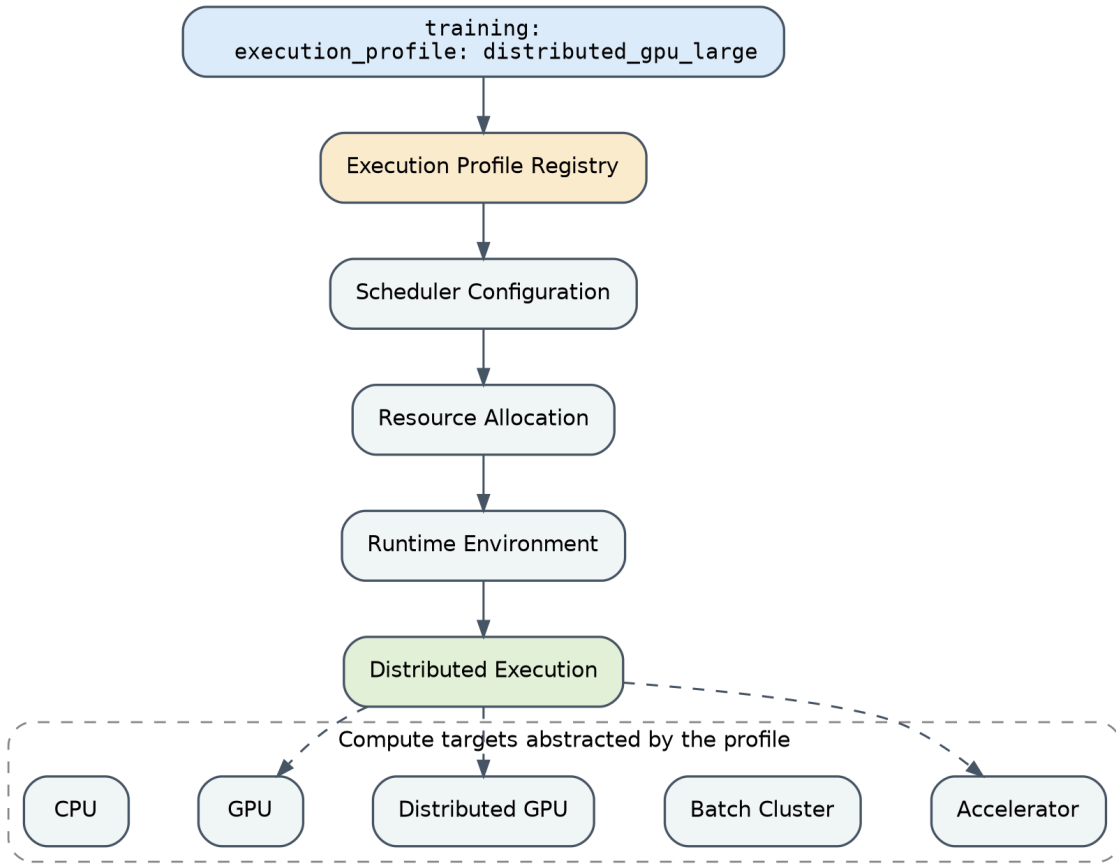


Figure 17. Execution profiles. A symbolic profile name in configuration expands to scheduler, resource, and runtime settings, abstracting the underlying compute targets.

A model owner writes `execution_profile: distributed_gpu_large` rather than configuring low-level scheduling infrastructure. Internally the profile expands to scheduler configuration, resource allocation, runtime environment, and distributed execution settings. This allows platform infrastructure to evolve independently from ML application code: when a new accelerator class becomes available, the platform team updates the profile registry and existing configurations benefit without modification.

19.1 Framework Adapters

Framework heterogeneity is handled by stage implementations rather than by the lifecycle. The training stage contract specifies inputs (dataset, feature set, code, hyperparameters, profile) and outputs (model artifact, metrics, checkpoints). Adapters for each supported framework implement that contract. The evaluation stage similarly relies on a model-loading adapter keyed by the framework recorded in the model namespace. New frameworks are added by contributing adapters; the lifecycle, policies, lineage, and observability are unchanged.

20. Scaling Characteristics

A conventional architecture scales approximately with the number of model-specific pipelines. Let N be the number of production models and P the number of independently implemented infrastructure components each model requires (data pipeline, training pipeline, evaluation, deployment, monitoring, and so on). Operational complexity in a pipeline-centric design then trends toward:

$$O(N \cdot P)$$

This is not a strict computational-complexity statement, but it captures the engineering relationship: more models generate more infrastructure artifacts, and each artifact must be built, tested, upgraded, and operated. Under a unified lifecycle platform, shared capabilities are implemented once and reused. Conceptually:

$$O(P + N \cdot C)$$

where C represents relatively lightweight model-specific configuration. As C becomes much smaller than P , the marginal infrastructure cost of introducing another model decreases significantly. Figure 18 illustrates the relationship for representative parameter values; the values are illustrative and are not measurements.

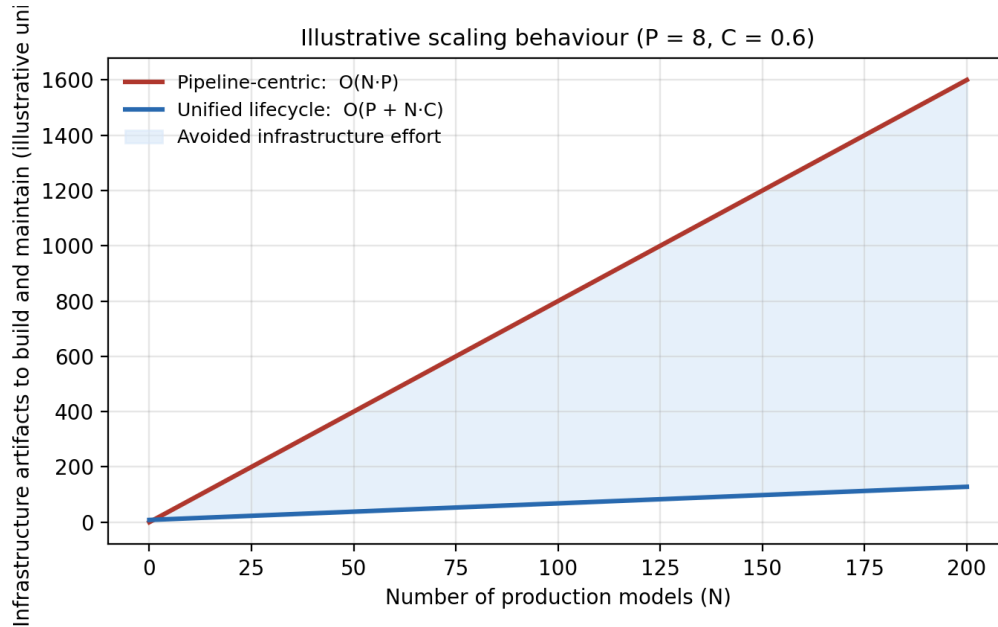


Figure 18. Illustrative scaling behavior of pipeline-centric versus unified lifecycle architectures as model count grows. Parameters are chosen for illustration only.

20.1 What Scales and What Does Not

The analysis above concerns infrastructure artifacts, not compute. Training compute scales with the models and data an organization chooses to train regardless of architecture; the platform does not reduce it, though idempotent recovery (Section 16) reduces waste. What the platform changes is the number of distinct things engineers must build and operate. The fixed term P grows slowly: new metaworkflow families, stage implementations, and adapters are added occasionally by the platform team. The per-model term C is bounded by the size of a configuration document and the model code itself.

20.2 Control-Plane Scaling

The control plane is the shared component whose load grows with N . Its load is measured in lifecycle events (stage dispatches, completions, lineage edges, policy decisions), which are small and infrequent relative to the compute they describe. A control plane designed as a metadata service with an append-only event log and indexed object store scales comfortably to thousands of models; the practical limits are in lineage query complexity for very deep graphs, which is addressed with materialized ancestry indexes.

21. Infrastructure Before and After Lifecycle Unification

Before unification, the number of operational artifacts increases continuously with model count, as shown earlier in Figure 1. After unification, the picture is the one shown in Figure 19: a control plane and metaworkflow engine at the top, model configurations in the middle, shared lifecycle services beneath, and lineage and governance underlying everything.

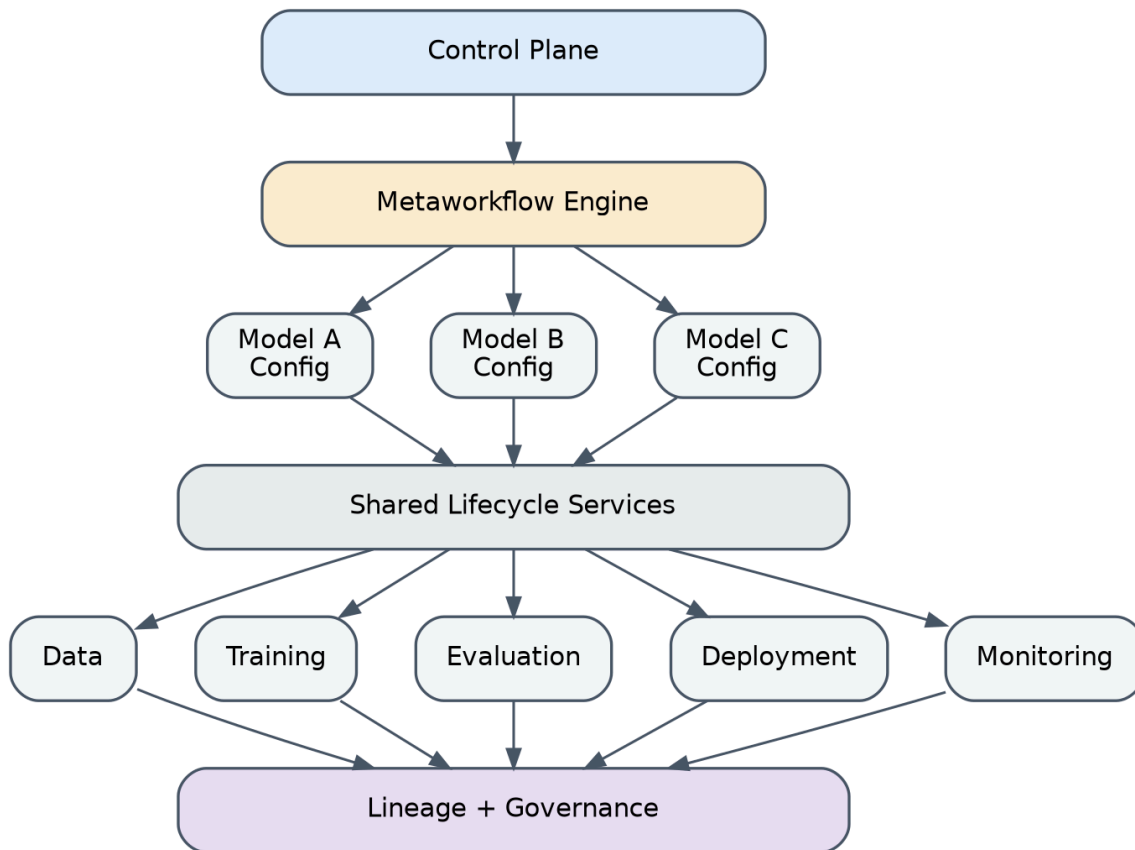


Figure 19. Infrastructure after lifecycle unification. Models are configurations of a shared lifecycle; data, training, evaluation, deployment, and monitoring are shared services; lineage and governance are common substrate.

The architecture changes from pipeline reuse, in which teams borrow fragments of one another's pipelines, to lifecycle reuse, in which teams share the lifecycle itself. The visible symptom is that adding Model D does not add a new box to the diagram; it adds a configuration.

22. Implementation Considerations

This section discusses practical concerns that arise when realizing the architecture. They are presented as design considerations rather than as a prescription for particular technologies.

22.1 Metadata Store

The control plane requires a metadata store with three properties: strong consistency for state transitions (a model version must not be both QUALIFIED and REJECTED), efficient graph traversal for lineage, and an append-only history for audit. A common realization pairs a transactional relational store for objects and state with a graph index or materialized ancestry table for lineage queries, and an event log that records every transition. Objects are immutable once they participate in lineage; mutation is expressed as creation of a new version.

22.2 Artifact Storage

Artifacts are stored in content-addressed object storage. The control plane stores references and hashes, not content. Content addressing provides the immutable identifiers that idempotent execution and validity checking depend on. Retention policies are expressed as governance rules and enforced by the control plane; an artifact referenced by a production or rollback-eligible model version cannot be garbage collected.

22.3 Multi-Tenancy and Isolation

A shared platform serves many teams. Isolation is required at three levels: data access (a lifecycle may only reference datasets its owner is permitted to read), compute (execution profiles map to quotas and namespaces), and metadata (lineage queries respect visibility). Because access control is part of the control plane, it is enforced at compile time and dispatch time rather than being left to individual pipelines.

22.4 Migration

Organizations adopt the architecture incrementally. A practical migration path is to introduce the control plane first as a system of record for existing pipelines (registering their outputs and capturing whatever lineage they can report), then to wrap existing pipelines as opaque stage implementations behind the SDK, then to migrate model families one at a time to metaworkflows. Each step delivers value independently: registration and lineage improve auditability before any pipeline is rewritten, and the SDK boundary reduces coupling before metaworkflows are adopted.

22.5 Escape Hatches

Some workloads will not fit any metaworkflow family. The platform should provide a custom-workflow path that still registers with the control plane, still emits lineage, and still passes qualification and deployment gates, but in which the graph is authored explicitly. The existence of this path is what allows the metaworkflow families to remain small; the requirement that it still passes through the gates is what prevents it from becoming a governance bypass.

23. Discussion

The architectural distinction between workflow orchestration and lifecycle management is important. A workflow engine answers: what task should execute next? A unified ML lifecycle platform must additionally answer: What model is being produced? Which data generated it? What is its current lifecycle state? What evaluation policy applies? Is it eligible for production? Which artifact replaced the previous model? Which production deployment uses it? Can the complete training process be reproduced? What should happen when the next dataset arrives?

These are machine-learning lifecycle semantics rather than generic workflow semantics. For this reason, lifecycle management should exist as an abstraction above orchestration. The platform may use different orchestration, storage, compute, registry, monitoring, and deployment technologies underneath it; those systems become implementation details behind stable lifecycle contracts. This produces the layered decoupling shown in Figure 20.

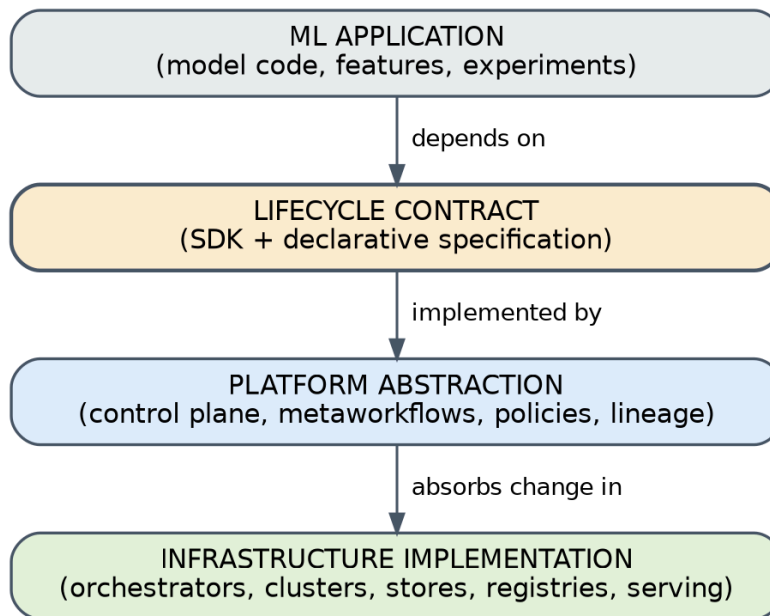


Figure 20. Layered decoupling. The ML application depends on the lifecycle contract; the platform abstraction implements the contract and absorbs infrastructure change.

The ML application depends primarily on the lifecycle contract. The platform absorbs infrastructure change. This is the same separation that operating systems provide between applications and hardware, and it has the same consequence: the application population can grow large and diverse while the platform beneath it evolves.

23.1 Organizational Implications

The architecture changes the division of labor. Model teams own model code, features, and configuration. The platform team owns metaworkflow families, stage implementations, policies, and the control plane. Governance functions own policy content but not enforcement mechanics. This division is only stable if the platform team treats configuration schema, SDK, and policy interfaces as products with compatibility commitments; a platform that changes its contracts freely recreates the coupling problem one level up.

23.2 Relationship to Agentic and LLM Workloads

Although this paper is framed around supervised model training, the lifecycle abstraction applies to other ML artifacts. A prompt template, a retrieval index, a fine-tuned language model, or an agent configuration each has versions, inputs, evaluations, deployments, and policies. Treating them as first-class objects in the same control plane, with evaluation policies appropriate to their modality, extends governance by construction to workloads whose evaluation is often less standardized than that of classical models.

24. Limitations and Future Work

The architecture has limitations that should be stated plainly.

- **Abstraction cost.** A metaworkflow family must be general enough to cover many models and specific enough to be useful. Getting this balance wrong produces either a family that needs constant extension or one that leaks infrastructure detail into configuration. The escape hatch (Section 22.5) mitigates but does not eliminate this tension.
- **Numerical reproducibility.** The platform guarantees specification and environment reproducibility but cannot guarantee bitwise identical parameters for all distributed workloads (Section 14.1).
- **Policy expressiveness.** Declarative policies capture threshold and comparison rules well. Judgment-based reviews (for example, of fairness in a novel domain) must be represented as approval gates whose content lives outside the platform.
- **Control-plane centrality.** The control plane is a shared dependency. Its unavailability blocks new lifecycle instances, though it does not affect models already serving. Availability engineering for the control plane is a prerequisite for adoption at scale.
- **Empirical validation.** This paper presents an architectural argument and an illustrative scaling model. Quantitative evaluation of engineering effort, incident rates, and time-to-production across pipeline-centric and lifecycle-centric organizations remains future work and is complicated by the difficulty of controlled comparison.

Future work includes formalizing the lifecycle contract as a typed specification amenable to static verification; extending qualification policies to cover multi-model systems whose behavior depends on interactions between components; and integrating cost accounting into the control plane so that lineage can attribute compute and storage cost to models, teams, and decisions.

25. Conclusion

This paper introduced a Unified Machine Learning Lifecycle Management Platform for operating large-scale production ML systems. The architecture addresses pipeline proliferation by moving the primary abstraction from individual workflows to a shared machine-learning lifecycle. Its core technical elements are:

1. Metaworkflows that generate model-specific execution graphs from reusable lifecycle templates.
2. Declarative configuration that separates model intent from infrastructure mechanics.

3. A stable SDK layer separating machine-learning code from orchestration and execution infrastructure.
4. Control-plane and execution-plane separation allowing lifecycle semantics to remain independent of compute implementation.
5. First-class lifecycle objects for datasets, models, experiments, evaluations, artifacts, and deployments.
6. End-to-end lineage connecting production models to their datasets, features, source code, parameters, runtime environments, and evaluation history.
7. Evaluation and qualification as platform primitives rather than model-specific release scripts.
8. Persistent lifecycle state and failure recovery supporting efficient retry and reproducible execution.
9. Governance by construction, in which quality, policy, lineage, and deployment requirements are automatically inherited by workloads.
10. Continuous model refresh, allowing production models to be retrained, compared, qualified, and promoted through the same governed lifecycle.

The fundamental architectural change can be summarized simply. Traditional production ML infrastructure follows: new model, new pipeline, new infrastructure complexity. A unified lifecycle architecture follows: new model, configuration, shared lifecycle platform. This changes the scaling model of machine-learning infrastructure. Instead of repeatedly building infrastructure around individual models, organizations invest in reusable lifecycle capabilities that can be applied across model classes, frameworks, teams, and execution environments.

The platform therefore goes beyond orchestration. It provides a control plane for the complete machine-learning lifecycle, defining how models are created, trained, evaluated, registered, deployed, observed, governed, reproduced, and continuously refreshed. As production ML systems continue to grow in scale and organizational importance, this lifecycle-centric architectural model provides a foundation for operating machine learning as a reliable and reusable engineering platform rather than a collection of independently maintained pipelines.

References

- [1]D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," in Advances in Neural Information Processing Systems (NeurIPS), 2015.
- [2]D. Baylor et al., "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform," in Proc. ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD), 2017.
- [3]J. Hermann and M. Del Balso, "Meet Michelangelo: Uber's Machine Learning Platform," Uber Engineering Blog, 2017.
- [4]J. Dunn, "Introducing FBLeaRner Flow: Facebook's AI Backbone," Facebook Engineering Blog, 2016.
- [5]Netflix, "Metaflow: A Framework for Real-Life Data Science," open-source release and documentation, 2019.

- [6]M. Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, 2018.
- [7]N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data Management Challenges in Production Machine Learning," in *Proc. ACM SIGMOD*, 2017.
- [8]M. Vartak et al., "ModelDB: A System for Machine Learning Model Management," in *Proc. Workshop on Human-In-the-Loop Data Analytics (HILDA)*, 2016.
- [9]L. Moreau and P. Missier (eds.), "PROV-DM: The PROV Data Model," *W3C Recommendation*, 2013.
- [10]E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction," in *Proc. IEEE Int. Conf. on Big Data*, 2017.
- [11]S. Amershi et al., "Software Engineering for Machine Learning: A Case Study," in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019.
- [12]A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in Deploying Machine Learning: A Survey of Case Studies," *ACM Computing Surveys*, vol. 55, no. 6, 2022.
- [13]D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine Learning Operations (MLOps): Overview, Definition, and Architecture," *IEEE Access*, vol. 11, 2023.